

```
import time

start_time = time.time()


epochs = 5

train_losses = []
test_losses = []
train_correct = []
test_correct = []


for i in range(epochs):

    trn_corr = 0
    tst_corr = 0
    train_loss = 0

    # ----- TRAIN -----
    for b, (X_train, y_train) in enumerate(train_loader):

        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)

        predicted = torch.max(y_pred.data, 1)[1]
        batch_corr = (predicted == y_train).sum()
        trn_corr += batch_corr

        optimizer.zero_grad()
        loss.backward()
```

```

optimizer.step()

train_loss += loss.item()

if b % 600 == 0:
    print(f'Epoch: {i} Batch: {b} Loss: {loss.item()}')

train_losses.append(train_loss / len(train_loader))
train_correct.append(trn_corr.item())

# ----- TEST -----
test_loss = 0

with torch.no_grad():
    for b, (X_test, y_test) in enumerate(test_loader):

        y_val = model(X_test)
        loss = criterion(y_val, y_test)

        predicted = torch.max(y_val.data, 1)[1]
        tst_corr += (predicted == y_test).sum()

        test_loss += loss.item()

test_losses.append(test_loss / len(test_loader))
test_correct.append(tst_corr.item())

```

```
# ----- TIME -----  
  
current_time = time.time()  
  
total = current_time - start_time  
  
print(f'Training Took: {total/60:.2f} minutes!')
```

```
import time  
start_time = time.time()  
  
# Create Variables to Tracks Things  
epochs = 5  
train_losses = []  
test_losses = []  
train_correct = []  
test_correct = []  
  
# For Loop of Epochs  
  
for i in range(epochs):  
    trn_corr = 0  
    tst_corr = 0  
    train_loss = 0  
  
# Train  
    for b,(X_train, y_train) in enumerate(train_loader):
```

```

    b+1 # start our batches at 1

    y_pred = model(X_train) # get predicted values form the training set. Not
    flattened

    loss = criterion(y_pred, y_train) # how off are we? Compare the predictions
    to correct answers in y_train


    predicted = torch.max(y_pred.data, 1)[1] # add up the number of correct
    prediction. Index off the first point

    batch_corr = (predicted == y_train).sum() # how many get got correct from
    this batch. True = 1, False=0, sum those up

    trn_corr += batch_corr # keep track as we go along in training


    # Update our parameter

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()


    # Print out some results

    if b % 600 == 0:

        print(f'Epoch: {i} Batch: {b} Loss: {loss.item()}')

train_losses.append(train_loss / len(train_loader))

train_correct.append(trn_corr.item())


# Test

```

```
with torch.no_grad(): #No gradient so we don't update our weights and biases with
test
    for b,(X_test, y_test) in enumerate(test_loader):
        y_val = model(X_test)
        predicted = torch.max(y_val.data, 1)[1] #Adding up correct predictions
        tst_corr += (predicted == y_test).sum() #T=1, F=0 and sum away

loss = criterion(y_val, y_test)
test_losses.append(loss)
test_correct.append(tst_corr)

current_time = time.time()
total = current_time - start_time
print(f'Training Took: {total/60} minutes!')
```